

NOTE:

This report is written for an expert viva panel. It is intentionally deep and includes detailed architecture, algorithms, data flow, and key code excerpts. All paths referenced are from the actual project structure.

=====

0. TABLE OF CONTENTS

=====

1. Project Overview
 2. System Architecture
 3. Frontend Deep Explanation
 4. Backend Deep Explanation
 5. Core Algorithms & Logic
 6. Simulation Lab Explained
 7. Data Source Explanation (Simulated vs Real)
 8. Real-World Mapping
 9. Industry Readiness Analysis
 10. Deployment in Real Industry
 11. Limitations & Improvements
 12. Viva Questions & Answers (40)
 13. Code Walkthrough (Key Files & Key Functions)
 14. Future Scope
- Appendix A: API Endpoints Summary
Appendix B: Database Schema Summary
Appendix C: Key Code Excerpts

=====

1. PROJECT OVERVIEW

=====

Project Title:

- AI-Based Identity-Centric Zero Trust Access Control Platform (CyberWatch Analytics UI)

Problem Statement:

- Legacy access models trust users after a single authentication. Modern attackers exploit stolen credentials, bypass MFA using phishing, and persist inside networks. A Zero Trust system must continuously verify identity, device, and behavior.

Objective:

- Build a full-stack platform that:
 - Learns a behavioral baseline per user
 - Computes risk scores on every login and session heartbeat
 - Enforces adaptive policies (allow, step-up, deny, lock)
 - Maps anomalies to MITRE ATT&CK techniques
 - Provides SOC dashboards for analysts

- Includes an attack simulation lab for validation

What It Solves in Cybersecurity:

- Detects risky logins using behavior deviations
- Detects credential abuse via login attempt frequency
- Enforces continuous session evaluation
- Provides traceable audit logs + MITRE mapping
- Enables SOC-level visibility and response simulation

Key Features (Mapped to Code):

- JWT auth + refresh + CSRF (backend/app/core/security.py, backend/app/api/v1/endpoints/auth.py)
- Role-Based Access Control (backend/app/api/deps.py, frontend/src/utils/roles.js)
- Baseline learning (backend/app/models/baseline.py, backend/app/services/baseline_service.py)
- Risk engine (backend/app/services/risk_engine.py)
- Anomaly engine (backend/app/services/anomaly_engine.py)
- Policy engine (backend/app/services/policy_engine.py)
- Session monitoring (backend/app/api/v1/endpoints/sessions.py + frontend polling)
- MITRE ATT&CK mapping (backend/app/services/mitre_service.py)
- Attack simulation engine (backend/app/services/simulation_engine.py + frontend/src/store/simulationEngine.js)
- SOC UI dashboard (frontend/src/pages/*)

2. SYSTEM ARCHITECTURE

High-Level Architecture (Textual Diagram):

[React + Vite Frontend] <-- REST + WebSocket --> [FastAPI Backend] <-- ORM --> [SQL Database]

Frontend Architecture:

- React + Vite + Tailwind CSS
- Page layout is driven by DashboardLayout (topbar + sidebar)
- Each page uses DashboardProvider state
- Charts and metrics are rendered with Recharts
- Page transitions via Framer Motion (MotionPage.jsx)

Backend Architecture:

- FastAPI app: backend/app/main.py
- Routers: backend/app/api/v1/endpoints
- Services: backend/app/services (business logic)
- Models: backend/app/models (SQLAlchemy ORM)
- Schemas: backend/app/schemas (Pydantic validation)

Communication:

- REST API calls via Axios (frontend/src/api/client.js)
- JWT access token in Authorization header
- Refresh token in HttpOnly cookie

- CSRF token validated on all mutating requests
- WebSocket /ws/events for live audit updates

Security Layers:

- Argon2 hashing for password storage
- Rate limiting for login & API
- CSP/HSTS/security headers
- RBAC enforcement at API boundary

=====

3. FRONTEND DEEP EXPLANATION

=====

Framework & UI stack:

- React: component-based, predictable state handling
- Vite: fast bundling & DX
- Tailwind: utility-first styling, consistent design system
- Recharts: visualization (risk, decisions, trends)
- Radix UI: accessible primitives
- Framer Motion: transitions

Folder Structure:

- frontend/src/pages: page-level views
- frontend/src/components: layout + shared components
- frontend/src/components/ui: UI primitives
- frontend/src/context: auth state
- frontend/src/store: dashboard state + simulation engine
- frontend/src/hooks: polling + WebSocket integration
- frontend/src/api: API client + token refresh
- frontend/src/utils: roles, fingerprint, time parser

Detailed Page Logic:

Executive Overview (frontend/src/pages/ExecutiveOverview.jsx)

- Computes riskScore from dashboard.effectiveRiskScore
- Trust Score = 100 - riskScore
- Uses PieChart for gauge
- System Health panel shows microservice statuses
- Recent alerts derived from threatIntel feed
- Quick actions navigate to other pages

Data Details (frontend/src/pages/DataDetails.jsx)

- Search across log fields (timestamp, username, action, message, IP, device)
- Risk filter + time range filter
- Client-side pagination (10 rows/page)
- Export button calls dashboard.actions.exportLogs (downloads CSV)

Predictive & Risk (frontend/src/pages/PredictiveRisk.jsx)

- Live risk score gauge (SVG arc)

- Decision distribution chart from decision breakdown
- Risk factors computed from logs + simulation factors
- Risk trend with AreaChart (auto-updates every 5 seconds)
- AI insights collapsible panel using liveRisk.insight

Behavioral Analysis (frontend/src/pages/BehavioralAnalysis.jsx)

- Displays active/high-risk/terminated sessions
- Session Monitor list with per-session actions
- Filter dropdown for risk levels
- "View Details" opens modal dialog
- Terminate action calls backend /sessions/terminate

Network & Relations (frontend/src/pages/NetworkRelations.jsx)

- Global network view (animated nodes + connections)
- MITRE view showing tactics grouped from riskAnalytics.mitre_techniques
- Threat feed (severity badges + timestamps)

Actionable Intelligence (frontend/src/pages/ActionableIntelligence.jsx)

- System metrics (latency, CPU, memory)
- Service health cards
- Settings panel bound to backend /settings

Simulation Lab (frontend/src/pages/SimulationLab.jsx)

- Shows simulation status, blocked attacks, mitigated threats, breaches
- Start/Pause/Reset buttons bind to dashboard.actions.startSimulation/stopSimulation/resetSimulation
- Timeline and Injected Attacks tabs
- Risk assessment progress bars
- Outcome summary with grade and recommendations

Frontend Data Flow (Global):

- AuthContext handles login/register/step-up -> tokens stored -> /auth/me
- DashboardProvider uses useDashboardData to fetch all datasets
- WebSocket pushes AuditLog events to state
- Simulation engine overlays synthetic logs and sessions

=====

4. BACKEND DEEP EXPLANATION

=====

Framework & Libraries:

- FastAPI
- SQLAlchemy
- Pydantic
- SlowAPI (rate limiting)
- Passlib Argon2 (password hashing)

Backend File Layout:

- backend/app/main.py: app init, middleware, exception handling
- backend/app/api/v1/endpoints/*.py: REST endpoints

- backend/app/services/*.py: business logic
- backend/app/models/*.py: ORM tables
- backend/app/schemas/*.py: request/response validation

Authentication Flow:

- Login: verify credentials -> run risk + anomaly -> decide policy
- If step-up -> issue OTP challenge
- If allow -> create session + issue tokens
- Admins are never locked (admin override logic)

Session Handling:

- AccessSession created on successful auth
- Heartbeat checks risk score periodically
- High/critical -> terminate session

Logging & Audit:

- Every auth/session/policy event stored in AuditLog
- WebSocket broadcasts log events to clients

5. CORE ALGORITHMS & LOGIC

Risk Engine (backend/app/services/risk_engine.py):

- Baseline deviation scoring
- Output: score, level, anomalies, MITRE mappings

Anomaly Engine (backend/app/services/anomaly_engine.py):

- Weighted scoring:
login_time 0.24, ip_change 0.22, device_change 0.18, session_duration 0.20,
login_attempt_frequency 0.16
- Combined risk formula: $\text{total} = 0.65 * \text{base} + 0.35 * \text{anomaly}$

Policy Engine:

- Low -> allow
- Medium -> step-up
- High -> deny
- Critical -> lock + alert

6. SIMULATION LAB EXPLAINED

Backend Simulation:

- In-memory state
- Steps every 3 seconds
- Updates riskScore and anomalyScore
- Writes logs into AuditLog
- If risk ≥ 85 -> "Access Blocked"

Frontend Simulation:

- Timeline events with MITRE tags
- Injected attack logs
- Session overlays for SOC realism

7. DATA SOURCE EXPLANATION

Because this project is not yet connected to real SOC telemetry:

- Logs are generated by authentication/session/simulation events
- Risk scores come from rule-based risk + anomaly engine
- System health metrics are simulated with jitter + DB latency

8. REAL-WORLD MAPPING

Production integrations would include:

- SIEM ingestion (Splunk, Elastic, Sentinel)
- IAM providers (Okta, Azure AD)
- Endpoint telemetry (EDR)
- Network telemetry (NetFlow, Zeek)
- Threat feeds (MISP, OTX)

9. INDUSTRY READINESS ANALYSIS

Is it industry-ready? Partially.

- The architecture is solid but missing production-grade telemetry ingestion, message queues, and ML detection.

10. DEPLOYMENT IN REAL INDUSTRY

Reuse: UI dashboards, policy engine, baseline logic

Replace: data ingestion, telemetry pipeline

Deploy: Kubernetes, PostgreSQL, Redis, Kafka

11. LIMITATIONS & IMPROVEMENTS

Limitations:

- Synthetic data
- No ML-based anomaly detection
- No distributed storage for simulation state

Improvements:

- ML risk engine
- Event pipeline

- Multi-tenant isolation

12. VIVA QUESTIONS & ANSWERS (40)

(See previous list; unchanged)

13. CODE WALKTHROUGH

(See key files list; unchanged)

14. FUTURE SCOPE

- AI risk models
- Real-time correlation
- Cloud-native scaling

APPENDIX A: API ENDPOINTS SUMMARY

- POST /auth/register
- POST /auth/login
- POST /auth/step-up
- POST /auth/refresh
- POST /auth/logout
- GET /auth/me
- GET /dashboard/overview
- GET /dashboard/profile
- GET /dashboard/logs
- GET /dashboard/risk-analytics
- GET /dashboard/threat-intel
- GET /dashboard/system-health
- GET /risk/live
- POST /risk/simulate-attack
- POST /risk/simulation/reset
- GET /sessions/active
- POST /sessions/heartbeat
- POST /sessions/terminate/<built-in function id>
- GET /settings
- PUT /settings
- GET /admin/users
- POST /admin/users/<built-in function id>/lock
- POST /admin/users/<built-in function id>/unlock
- GET /admin/logs/export
- POST /simulation/start
- POST /simulation/stop

- POST /simulation/reset
- GET /simulation/status

APPENDIX B: DATABASE SCHEMA SUMMARY

User:

- id, username, email, role, hashed_password, is_active, is_locked, failed_login_attempts, lock_until

AccessSession:

- id, user_id, ip_address, device_fingerprint, issued_at, expires_at, is_active, current_risk_score, current_risk_level

BehaviorBaseline:

- user_id, average_login_hour, known_locations, known_device_fingerprints, ip_history, access_frequency_per_day

AuditLog:

- id, timestamp, event_type, action, risk_score, risk_level, decision, mitre_technique_id, ip_address, message

BehaviorAnomalyEvent:

- user_id, session_id, anomaly_score, total_risk_score, risk_level, factors, metrics, detected_at

AppSettings:

- risk thresholds, monitoring flags, session monitor interval

APPENDIX C: KEY CODE EXCERPTS (PSEUDO FROM ACTUAL CODE)

Risk Scoring (risk_engine.py):

```
score = 0
if baseline missing: score += 8
if unusual login time: score += 10..25
if new location: score += 20
if new device: score += 25
if new ip: score += 15
if access freq high: score += 18
if failed attempts >=2: score += 20
if protocol != https: score += 12
if phishing simulated: score += 30
score = clamp(0..100)
```

Combined Risk (anomaly_engine.py):

```
combined = 0.65*base_risk + 0.35*anomaly_score
if anomaly_score > 70 and level in low/medium -> promote to high
```


Simulation Steps (simulation_engine.py):

Step1: Suspicious Login (+30 risk)

Step2: Brute Force (+35 risk)

Step3: Privilege Escalation (+25 risk)

If risk >= 85 -> block session

=====

DIRECT ANSWERS TO YOUR QUESTIONS

=====

1) Is the project industry-ready?

- Partially. Architecture is strong but requires production telemetry and scaling.

2) If industry wants to use it, what parts will they take?

- UI dashboards, policy engine, anomaly logic. They must integrate real data pipelines.

3) What does the project do in real life?

- Continuous identity verification + behavioral risk scoring + adaptive access enforcement.

4) Where do risk scores come from if no real data?

- From simulated telemetry + rule-based scoring on login context and baseline deviation.

=====

END OF REPORT

=====